

**B.M.S. COLLEGE OF ENGINEERING BENGALURU**

Autonomous Institute, Affiliated to VTU



Object Oriented Modelling Mini Project Report

**Public Transit System**

*Submitted in partial fulfillment for the award of degree of*

Bachelor of Engineering in  
Computer Science and Engineering

*Submitted by:*

**Ayush Ranjan (1BM23CS058)**

**Anshuman Gupta (1BM23CS043)**

**Ayaan Shrestha (1BM23CS056)**

**Annas Sharieff (1BM23CS041)**

Department of Computer Science and Engineering  
B.M.S. College of Engineering,  
Bull Temple Road, Basavanagudi, Bangalore 560 019  
2024-2025

**B.M.S. COLLEGE OF ENGINEERING**  
**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING**



***DECLARATION***

We, **Ayush Ranjan (1BM23CS058)**, **Anshuman Gupta (1BM23CS043)**, **Ayaan Shrestha (1BM23CS056)**, **Annas Sharieff (1BM23CS041)** students of 5<sup>th</sup> Semester, B.E, Department of Computer Science and Engineering, BMS College of Engineering, Bangalore, hereby declare that, this Object Oriented Modelling Mini Project entitled" **Public Transit System**" has been carried out in Department of CSE, B.M.S. College of Engineering, Bangalore during the academic semester Aug 2025-Dec 2025. I also declare that to the best of our knowledge and belief, the OOM mini Project report is not from part of any other report by any other students.

**Signature of the Candidates**

**Ayush Ranjan (1BM23CS058)**  
**Anshuman Gupta (1BM23CS043)**  
**Ayaan Shrestha (1BM23CS056)**  
**Annas Sharieff (1BM23CS041)**

**B.M.S. COLLEGE OF ENGINEERING DEPARTMENT OF  
COMPUTER SCIENCE AND ENGINEERING**



***CERTIFICATE***

This is to certify that the **Object-Oriented Modelling Mini Project** titled “**Public Transit System**” has been carried out by **Ayush Ranjan (1BM23CS058)**, **Anshuman Gupta (1BM23CS043)**, **Ayaan Shrestha (1BM23CS056)**, **Annas Sharieff (1BM23CS041)** during the academic year 2025-2026.

Signature of the Faculty in Charge

## Table of Contents

| S. No | Title                                    | Page No |
|-------|------------------------------------------|---------|
| 1     | Ch 1: Problem statement                  | 1-2     |
| 2     | Ch 2: Software Requirement Specification | 3-6     |
| 3     | Ch 3: Class Diagram                      | 7-9     |
| 4     | Ch 4: State Diagram                      | 10-12   |
| 5     | Ch 5: Interaction diagram                | 13-21   |
| 6     | Ch 6: UI Design with Screenshots         | 22-28   |

## Chapter 1: Problem Statement

### Public Transit System

Public transport services in many cities face operational inefficiencies due to the lack of real-time visibility of buses and their movement. Commuters often encounter unpredictable waiting times, missed buses, and overcrowded stops, reducing their trust in public transportation. Transport authorities and operators struggle to monitor fleet performance, manage delays, and respond to route disruptions effectively. The use of a Real-Time Public Transport Tracking System aims to address these challenges by providing accurate, live information on bus locations, arrival times, and service conditions.

#### Target Audience

The main users of this system include commuters who rely on timely and reliable public transport, bus operators managing fleet movement, and transport authorities responsible for monitoring service quality. Indirectly, it benefits the wider community by promoting efficient mobility, reducing traffic congestion, and improving travel safety.

#### Problems to Solve

- **Uncertain Wait Times:** Commuters often wait without knowing when the next bus will arrive, leading to frustration and reduced confidence in the system.
- **Fleet Monitoring Challenges:** Operators lack visibility into real-time bus positions, making it difficult to detect delays, breakdowns, or deviations.
- **Operational Inefficiency:** Manual tracking methods slow down decision-making, hinder quick corrective actions, and create communication gaps between operators and passengers.
- **Disruption Communication:** Commuters are frequently unaware of sudden route changes or delays due to the absence of a centralized alert system.
- **Resource Management:** Poor data availability prevents authorities from optimizing routes, managing peak-hour loads, or planning future improvements.

## **Operational Context**

The system will be deployed across transport hubs, bus depots, control centers, and mobile applications used by commuters. PTS integrates with GPS-enabled bus units, municipal databases of routes and stops, and communication networks. It is required during daily commuting hours, peak travel times, festival seasons, and any scenario where real-time visibility of public transport enhances safety and reliability.

## **Purpose**

PTS plays a crucial role in modernizing public transportation by offering transparency, predictability, and data-driven insights. It supports commuters by helping them plan their journeys more efficiently and reduces travel-related stress. For operators, it enables proactive decision-making, timely interventions, and improved resource allocation. Ultimately, the system enhances public satisfaction, increases the usage of public transport, and supports sustainable urban mobility.

## **Mechanism**

The system collects, processes, and displays real-time GPS data from buses to deliver accurate positional and timing information to users. The process involves:

- **Data Capture:** GPS devices on buses transmit location and speed data at regular intervals.
- **Processing:** The backend system analyzes incoming telemetry, updates bus positions, and calculates dynamic ETAs using route and traffic information.
- **Display & Alerts:** The processed data is shown on mobile and web applications through maps, lists, and notifications.
- In addition, it provides:
- **Low-Bandwidth Optimizations:** Ensures usability even in regions with poor mobile connectivity.
- **Data Security:** Protects user information and operational data through encryption and secure APIs.
- **Scalability & Reliability:** Supports large user bases and multiple city deployments while maintaining system performance.

## **CHAPTER 2: SOFTWARE REQUIREMENT SPECIFICATION**

### **PUBLIC TRANSIT SYSTEM**

#### **Problem Statement**

In small cities and tier-2 towns, commuters lack real-time visibility into public transport (buses), leading to uncertain wait times, overcrowding, and reduced trust in public transport. Operators lack tools to monitor fleet performance and proactively manage delays. PTS addresses this by delivering GPS-based live tracking, reliable ETAs, and low-bandwidth optimizations for accessibility.

#### **1. Introduction**

##### **1.1 Purpose of this Document**

This SRS describes the Public Transportation System (PTS) in detail. It defines objectives, stakeholders, functional and non-functional requirements, constraints, and high-level design. It also specifies models and behavioral views (use cases, states, sequences, activity) to guide implementation and testing.

##### **1.2 Scope of this Document**

PTS provides real-time bus tracking, ETAs per stop, route discovery, disruption alerts, and operator dashboards. It is designed for low-bandwidth environments and supports multi-tenant municipal deployments.

##### **1.3 Overview**

PTS comprises a mobile/web client, an application server with ETA and alert logic, a data pipeline to ingest GPS positions, and a persistence layer for users, routes, stops, telemetry, and events. Operator dashboards expose fleet KPIs and controls.

#### **2. General Description**

Primary Users and Roles:

Commuter: discover routes, track buses, view ETAs, receive alerts.

Bus Operator: update bus status, investigate GPS issues, broadcast service notices.

Administrator: configure routes/stops, manage fleets, compliance and audit.

#### Assumptions and Dependencies:

Buses carry GPS units or smartphones capable of periodic telemetry.

Mobile connectivity is intermittent; the system must degrade gracefully.

Municipal authority provides base route and stop data.

### 3. Functional Requirements

| ID   | Requirement                                                  |
|------|--------------------------------------------------------------|
| FR1  | User Registration & Login (email/phone OTP)                  |
| FR2  | Search & View Bus Routes and Stops                           |
| FR3  | Real-Time Tracking of selected routes/buses on map           |
| FR4  | ETA computation per stop with continuous refresh             |
| FR5  | Disruption & Delay Notifications (push/SMS/IVR fallback)     |
| FR6  | Operator Console: update bus status, publish alerts          |
| FR7  | Admin: CRUD routes, stops, buses, drivers, shifts            |
| FR8  | Low-Data Mode: compressed tiles, reduced polling             |
| FR9  | Offline Cache & Last Known Location                          |
| FR10 | Analytics: on-time performance, headway variance, dwell time |

### 4. Interface Requirements

Mobile/Web UI with map, list, and accessibility-first layouts.

REST/GraphQL APIs for clients; secure operator/admin portals.

GPS Telemetry Ingest API (authenticated, batched).

### 5. Performance Requirements

ETA refresh latency  $\leq 5$  seconds for active routes under normal load.

Support 1,000+ concurrent users per city node.

Server-side p95 API response time  $\leq 800$  ms; availability  $\geq 99.0\%$ .



## **6. Design Constraints**

Must operate under low bandwidth (2G/Edge) with graceful degradation.

Use encrypted transit and storage for sensitive data (TLS, at-rest encryption).

Geo stack compatible with open map data; time-series store for telemetry.

## **7. Non-Functional Attributes**

Security: RBAC, token-based auth, PII minimization, rate limiting.

Reliability: resilient ingest with retry/backoff; at-least-once processing.

Scalability: horizontal scaling of stateless services; sharded telemetry.

Usability: simple, low-clutter UI, large touch targets, dark/light modes.

Portability: Android/iOS/Web; containerized backend; cloud/on-prem ready.

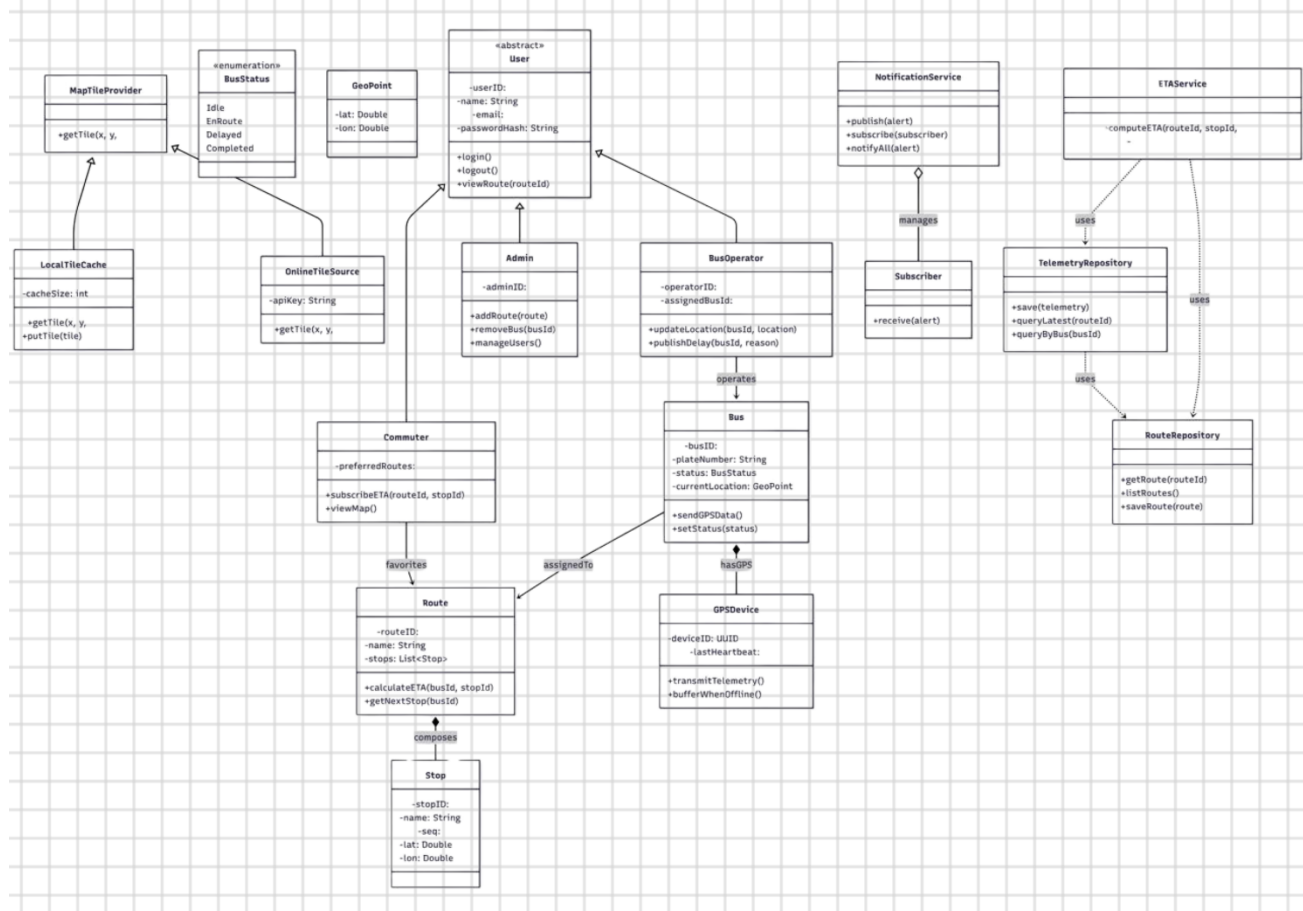
## **8. Preliminary Schedule and Budget**

Timeline: 5 months (Requirements 3w, Design 4w, Build 8w, Test 4w, Pilot 3w, Rollout 2w).

Team: 1 PM, 2 FE, 2 BE, 1 Mobile, 1 QA, 1 DevOps.

Budget: ₹35–45 lakhs (incl. infra and pilot ops).

## **CHAPTER 3: CLASS MODELING**



This class diagram represents the architecture of a Public Transportation System (PTS). It shows all major entities in the system, their attributes, behaviors, and how they interact to provide live bus tracking, ETA calculation, notification services, route management, and GPS telemetry handling.

## 1. User Hierarchy

At the center of the design is an abstract User class from which all types of users inherit. This supports common authentication and access functionality.

- User (Abstract Class)
  - Attributes: userID, name, email, passwordHash

- Methods: login(), logout(), viewRoute(routeId)

Three specific user roles extend this class:

- Admin
  - Manages system configuration
  - Methods: addRoute(), removeRoute(), manageUsers()
- BusOperator
  - Responsible for updating bus movement
  - Methods: updateLocation(), publishDelay()
- Commuter
  - End user viewing routes and ETAs
  - Methods: subscribeETA(), viewMap()

Can maintain preferred or favorite routes

This hierarchy supports role-based functionality while reusing common features.

## **2. Bus and GPS System**

- Bus
  - Represents each bus in the fleet.
  - Attributes: busID, plateNumber, status, currentLocation: GeoPoint
  - Methods: sendGPSData(), setStatus()
- GPSDevice
  - Each bus has a GPS device, responsible for sending telemetry.
  - Methods: transmitTelemetry(), bufferWhenOffline()
- BusStatus (Enumeration)
  - Defines bus states such as:

- Idle
- EnRoute
- Delayed
- Completed

This helps accurately track the live condition of each bus.

- GeoPoint
  - Represents coordinates (latitude & longitude).
  - Used to store the location of buses and stops.

### **3. Route and Stop Structure**

- Route
  - Each route contains:
    - Route identifier
    - Route name
    - List of associated stops
  - Methods to calculate ETA and find the next stop
- Stop
  - Each stop has:
    - stopID, name, sequence number
    - Latitude and longitude
  - A Route composes many Stops, meaning stops cannot exist without the route they belong to.
  - Relationships:
    - A Bus is assigned to a Route
    - A Commuter can mark a Route as favorite

### **4. Telemetry and Data Management**

- TelemetryRepository
  - Stores and retrieves GPS data.
  - save(telemetry)

- queryLatest(routeId)
- queryByBus(busId)
- RouteRepository
  - Manages route data.
  - getRoute(), listRoutes(), saveRoute()
  - These repositories act as the data layer of the system.

## 5. ETA Computation

- ETAService
- Computes the Estimated Time of Arrival for any bus at any stop.
- Method: computeETA(routeId, stopId)
  - Depends on:
    - TelemetryRepository (for bus location)
    - RouteRepository (for route and stop information)

## 6. Notification System (Observer Pattern)

- NotificationService:
  - Central component for alerts and updates.
  - Methods: publish(alert), subscribe(), notifyAll()
  - Manages subscribers interested in alerts.
- Subscriber:
  - Any user who receives notifications.
  - Method: receive(alert)
  - This implements an Observer Pattern where commuters can subscribe to ETA or delay alerts for specific routes.

## 7. Map Rendering Components

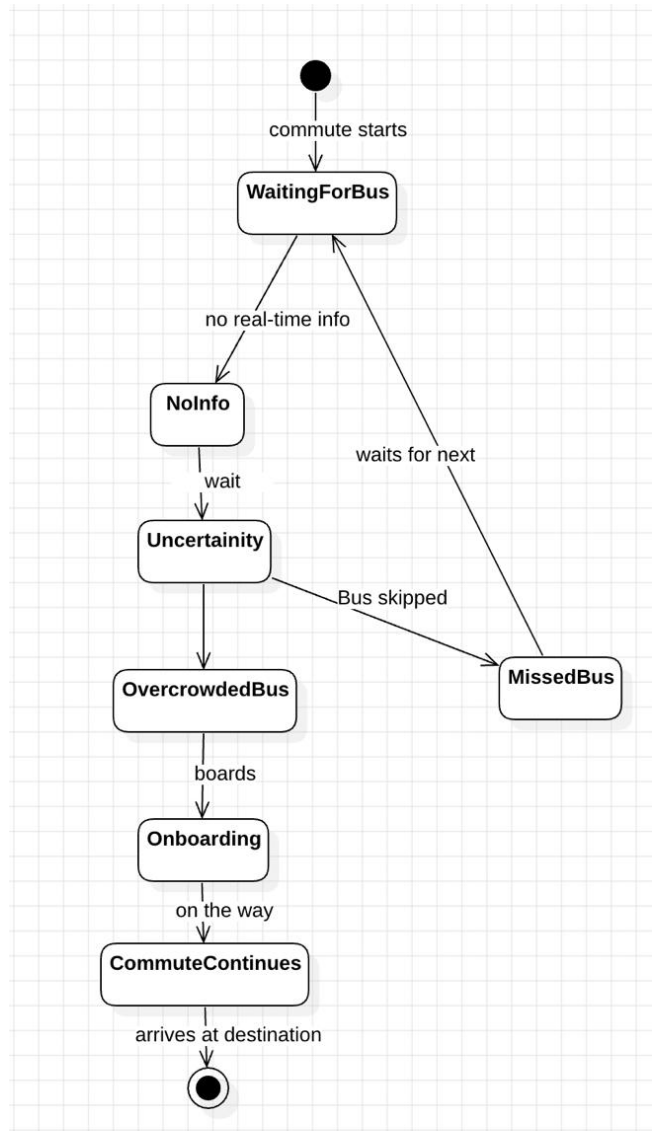
These components handle the display of map tiles in the commuter app.

- MapTileProvider: Abstract provider to fetch map tiles.
- OnlineTileSource: Fetches tiles from an internet map service (uses an API key).
- LocalTileCache: Caches tiles locally to improve speed and reduce network usage.

## Chapter 4: State Modeling

### 1. Commuter Journey State Model :

- This state model illustrates the real-world commuting experience *before* a Public Transportation System exists. It represents the uncertainty, lack of information, and inefficiencies that commuters commonly face.
- The diagram models the different states a commuter transitions through from the moment their journey begins until reaching the destination. The process starts at *WaitingForBus*, where the commuter lacks real-time visibility of bus arrival times. Without information, the commuter enters a *NoInfo* state that leads to *Uncertainty*—not knowing when the bus will arrive or how crowded it may be.
- This uncertainty diverges into two outcomes:
  - **OvercrowdedBus:** The bus arrives but is too full, forcing difficult decisions and uncomfortable onboarding.
  - **MissedBus:** The commuter fails to board because the bus was skipped, overcrowded, or arrived unexpectedly.
- If the commuter boards, the system transitions to *Onboarding* and then *CommuteContinues* as the commuter travels towards the destination. The state machine ends upon arrival.

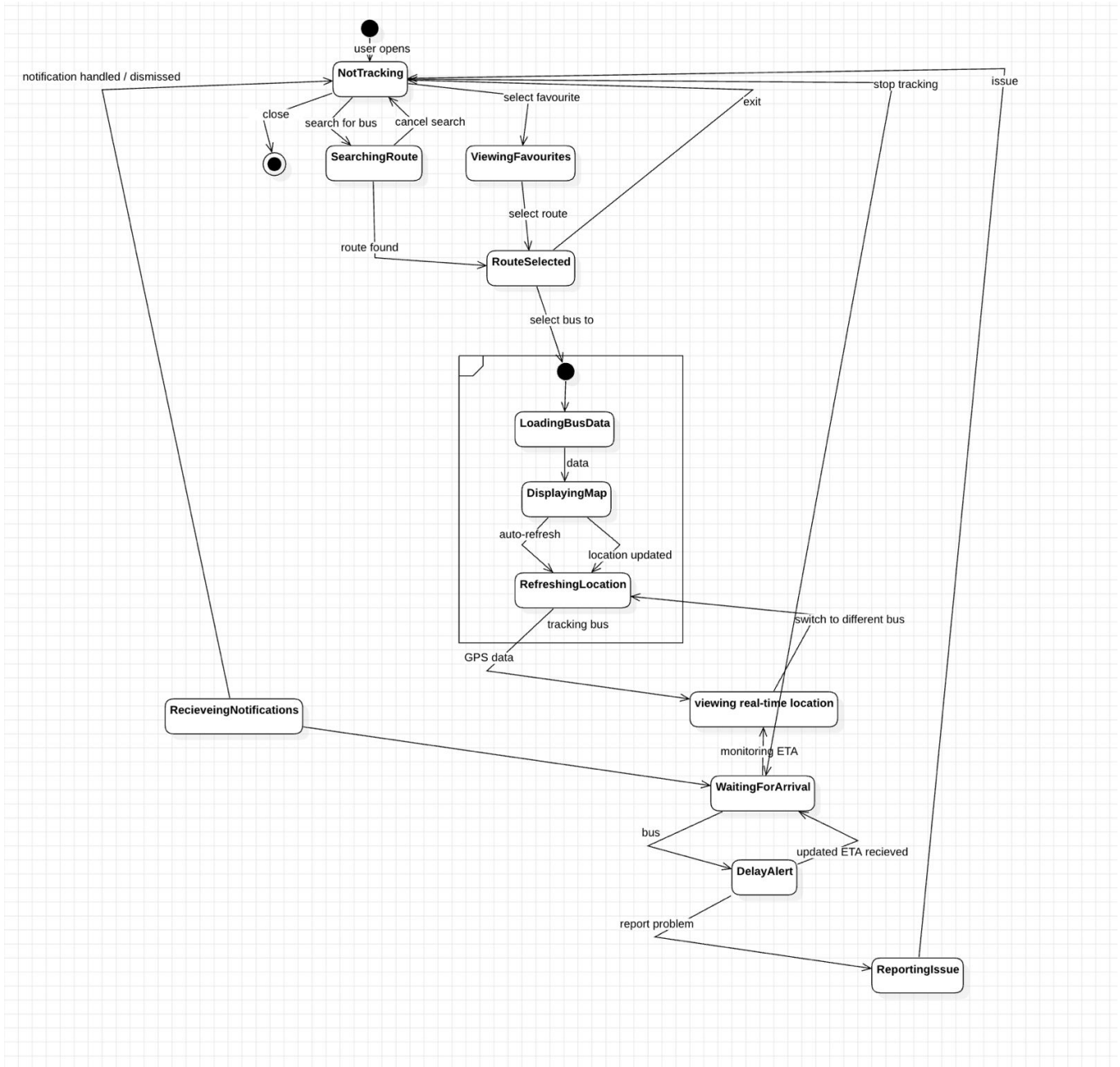


## 2. Commuter Tracking State Model

- This state model explains how a commuter interacts with the real-time tracking application once the system is implemented.
- This model begins when the user opens the application, initially entering the *NotTracking* state. From here, the commuter can search for a route (*SearchingRoute*) or browse favorites (*ViewingFavorites*). Once a route is selected, the user transitions to *RouteSelected* and then chooses a bus to monitor.
- The main functional state is *TrackingBus*, which includes:
  - LoadingBusData – fetching telemetry

- DisplayingMap – showing bus position on the map
- RefreshingLocation – periodically updating the bus location through GPS

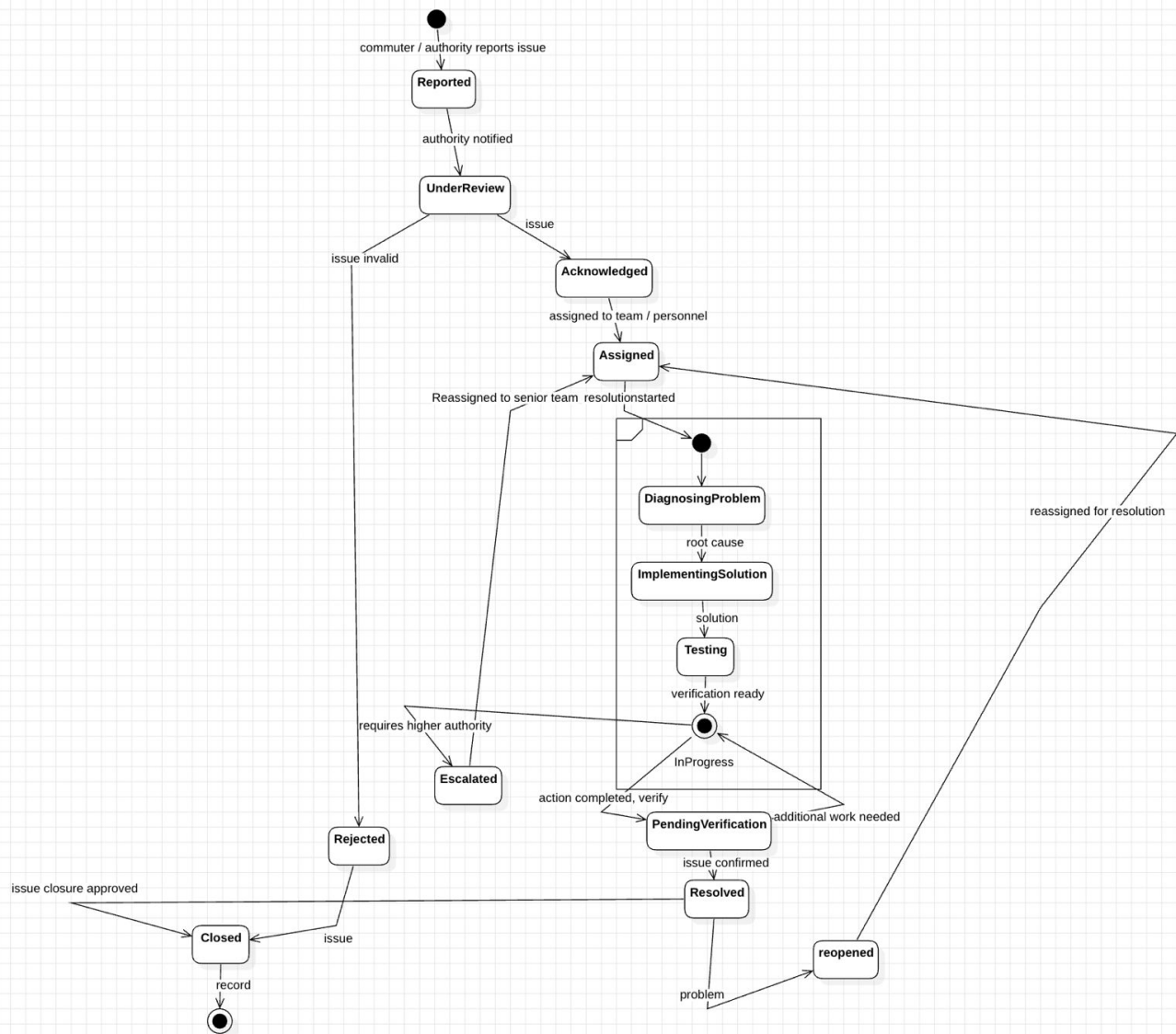
The commuter can monitor ETA, view real-time location, and respond to delays. The model accommodates actions such as switching buses, receiving notifications, or reporting issues. The tracking session ends when the user stops tracking or the bus arrives at the destination.





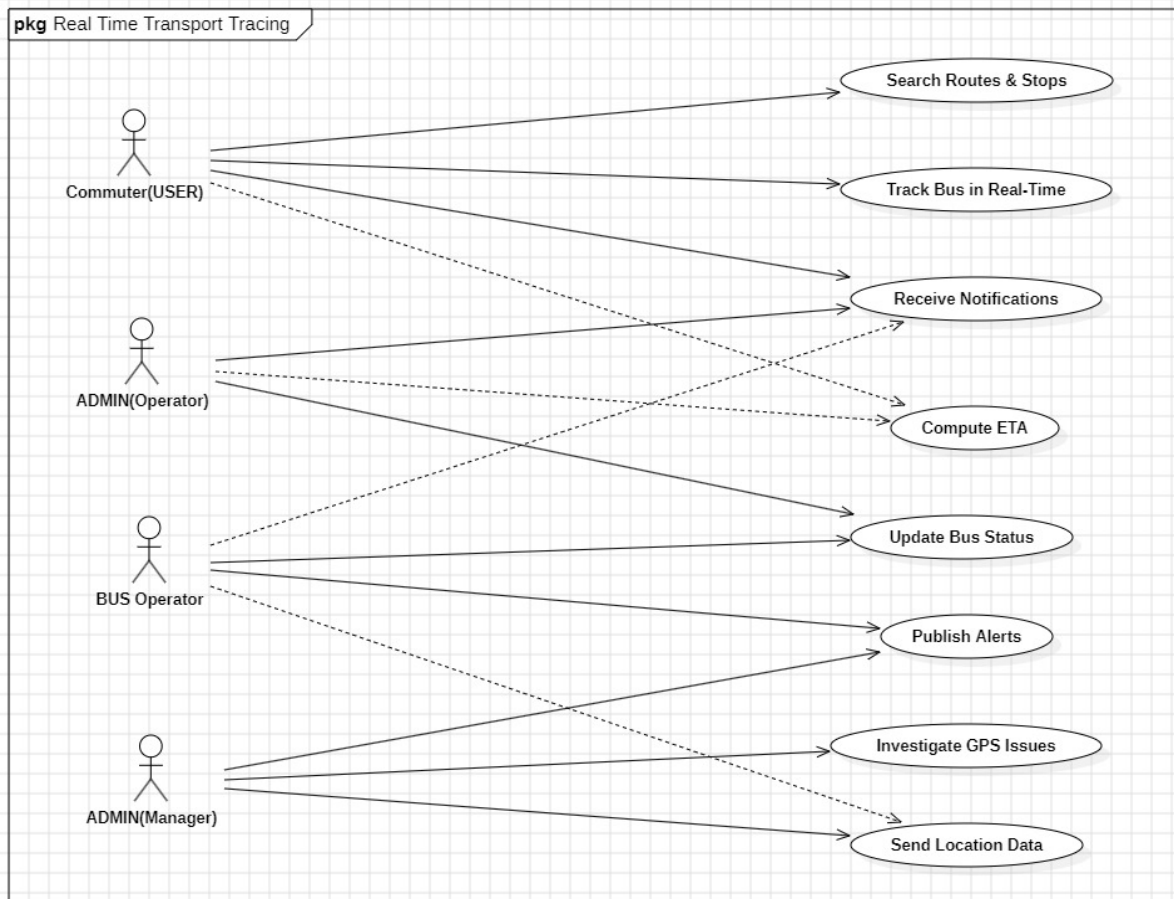
### 3. Grievance / Issue Resolution State Model

- This state model describes how user-reported issues (delays, GPS errors, wrong ETA, etc.) flow through the transport authority's grievance-handling mechanism.
- When a commuter or authority reports an issue, the system enters the *Reported* state. The issue then moves to *UnderReview*, where authorities verify its validity.
- If valid, the issue becomes *Acknowledged* and is *Assigned* to a responsible team. The system then transitions into the core state: *InProgress*, which contains a series of nested states:
  - DiagnosingProblem
  - IdentifyingRootCause
  - ImplementingSolution
  - Testing
- After corrective action, the issue enters *PendingVerification*. If verified successfully, the issue becomes *Resolved* and finally *Closed*. If additional work or higher authority is needed, the issue is *Reassigned* or *Escalated*. Additionally, a previously resolved issue may be *Reopened* if the problem reappears.



## Chapter 5: Interaction Modeling

### USE CASE DIAGRAM



The Use Case Diagram illustrates how different actors interact with the Public Transportation System (PTS). It captures the major functionalities provided to commuters, operators, administrators, and GPS devices. Each use case represents a specific goal the user wants to accomplish within the system.

#### **1. Actors**

##### **1. Commuter (User)**

Regular passengers who use the system to plan and track their commute.

##### **2. Bus Operator**

Transport staff responsible for monitoring buses, updating statuses, and ensuring fleet movement accuracy.

### **3. Admin**

Authority-level personnel who configure, manage, and monitor the overall system.

### **4. Bus GPS Unit**

An automated actor (device) that sends the bus's real-time location to the backend system.

## **2. Use Cases**

Below are all the use cases in the diagram with detailed relevance.

- **Register & Login (Commuter)**

Allows new commuters to create an account and existing commuters to log in.

**Relevance:**

Enables personalized features such as saving favorite routes, receiving notifications, and subscribing to ETAs.

- **Search Routes & Stops**

Commuters search for available bus routes, stops, and service information.

**Relevance:**

This use case is a core functionality since commuters must know which route or stop they want to track. It reduces confusion and supports journey planning.

- **Track Bus in Real-Time (Commuter & Operator)**

Displays the live location of a selected bus on a map.

**Relevance:**

- ✓ Helps commuters plan boarding time
- ✓ Helps operators monitor fleet movement
- ✓ Reduces uncertainty and waiting time
- ✓ Improves overall commuter satisfaction

This is the **central use case** of RTPTS.

- **Receive Notifications (Commuter)**

Commuters receive push notifications or alerts about delays, bus arrival, or service

disruptions.

**Relevance:**

Ensures the commuter is informed even when not actively tracking the bus. Enhances usability and reliability.

- **Compute ETA (System)**

The system calculates the Estimated Time of Arrival for a bus approaching a particular stop.

**Relevance:**

ETA is a critical feature for real-time tracking. The system uses historical data, GPS telemetry, and route speed to provide accurate predictions.

- **Update Bus Status (Operator)**

Operators can update the current condition of the bus (e.g., delayed, breakdown, diverted).

**Relevance:**

Keeps system data accurate. Helps commuters and admins see the real-time operational health of each bus.

- **View Fleet Dashboard (Admin)**

Admins get an overview of all buses, routes, active issues, and performance metrics.

**Relevance:**

Enables transport authorities to oversee operations, spot issues early, and take corrective actions.

- **Manage System Data (CRUD) (Admin)**

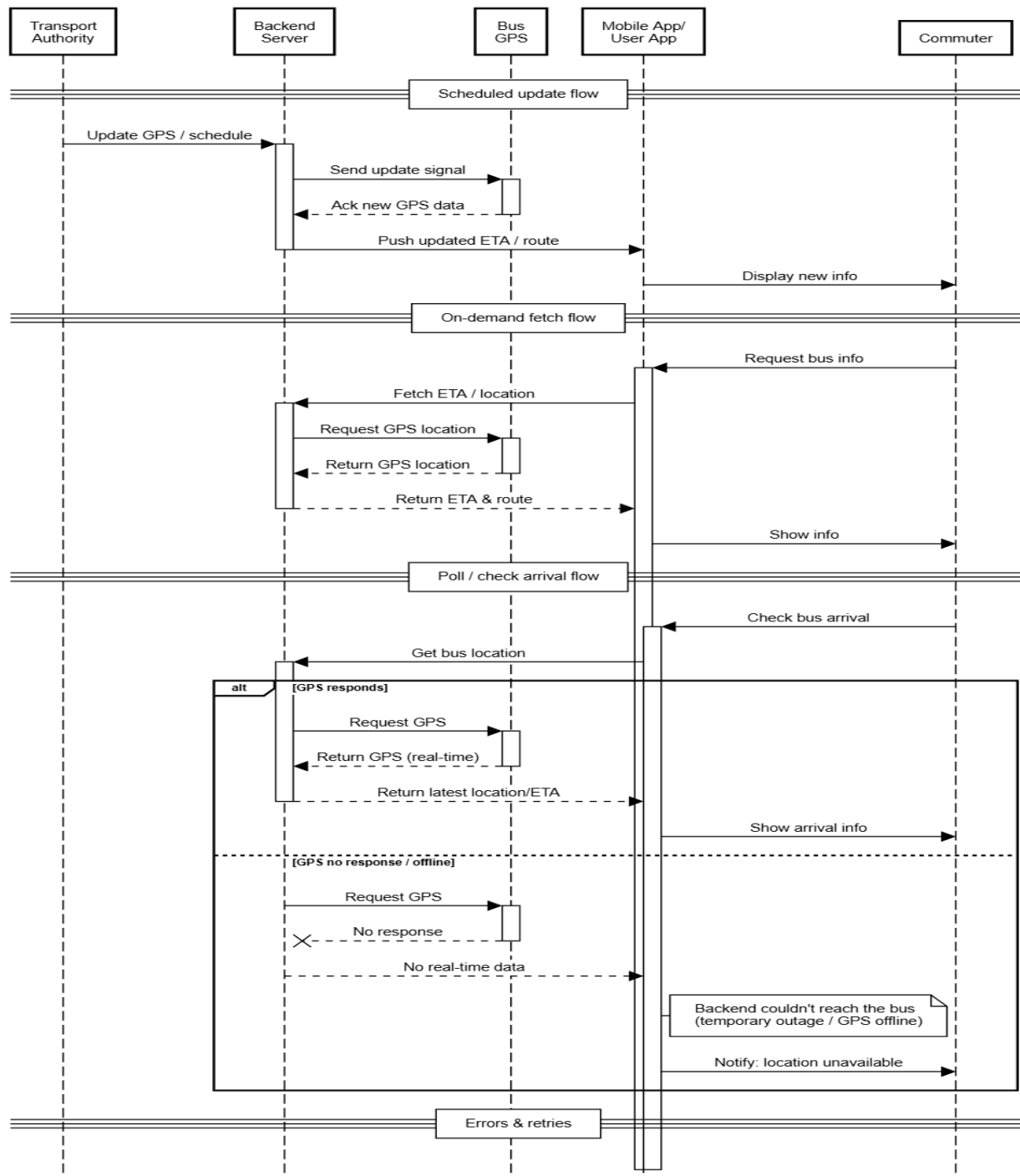
Admin can Create, Read, Update, Delete:

- Bus details
- Route information
- Stop lists
- User roles

**Relevance:**

Allows continuous update of system data, ensuring accurate route and fleet information.

## Sequence Diagram:



This sequence diagram describes the dynamic interaction among the principal components of the Public Transportation System (PTS). It details the flow of messages exchanged between the Transport Authority, Backend Server, Bus GPS Unit, Mobile Application, and Commuter during real-time bus tracking and ETA retrieval processes.

The diagram further illustrates the handling of exceptional conditions such as unavailable GPS data and network errors.

## **2. Participating Entities**

### **2.1 Transport Authority**

Initiates configuration updates related to bus schedules, route structures, and GPS parameters.

### **2.2 Backend Server**

Central processing unit responsible for data integration, ETA computation, and communication with GPS devices and user applications.

### **2.3 Bus GPS Unit**

Embedded hardware device that records and transmits location telemetry to the backend system.

### **2.4 Mobile App/User Application**

Client-side interface used by commuters to request bus information, receive updates, and review ETA data.

### **2.5 Commuter**

The end-user interacting with the application to obtain real-time information for transportation planning.

## **3. Normal Operational Flow**

### **3.1 Configuration Update**

- The Transport Authority transmits updated GPS, route, or schedule information to the Backend Server.
- The Backend Server acknowledges this update and propagates the necessary configuration changes to the Bus GPS Unit.

### **3.2 GPS Update Acknowledgment**

- The Bus GPS Unit receives the update signal and acknowledges receipt, ensuring synchronization between field hardware and backend parameters.

### **3.3 Dissemination of Updated ETA/Route Data**

- The Backend Server computes updated ETA and routing information using the most recent telemetry and operational data.

- The updated information is pushed to the Mobile Application for display to the commuter.

### **3.4 Commuter-Initiated Request for Bus Information**

- Upon receiving new information or by manual request, the commuter queries the Mobile Application for real-time bus status.
- The Mobile Application forwards this request to the Backend Server.

### **3.5 Retrieval of GPS and ETA Information**

- The Backend Server issues a location request to the Bus GPS Unit.
- The Bus GPS Unit returns current telemetry including latitude, longitude, speed, and

## **4. Periodic Polling and Monitoring**

- The commuter may continuously monitor bus arrival by repeatedly requesting updates.
- The Mobile Application continues to fetch bus location data from the Backend Server.
- The Backend Server interacts with the Bus GPS Unit as necessary to obtain the latest telemetry.

## **5. Exceptional and Error Handling Scenarios**

### **5.1 GPS Device Unavailability**

- If the Backend Server requests GPS data and receives no response, the system transitions to a *No Real-Time Data* condition.
- The Mobile Application notifies the commuter that real-time location data is unavailable.

### **5.2 Backend/Network Failure**

- In cases where the commuter requests real-time information and the Backend Server encounters network or internal failures, it responds with an error condition.
- The Mobile Application displays an error message indicating that the requested data could not be retrieved.



```

graph LR
    subgraph Authority
        A1[Update Schedules] --> A2[Update Bus GPS]
        A2 --> A3((PUSH UPDATES TO BACKEND))
    end

    subgraph App_Backend [APP BACKEND]
        A3 --> B1(( ))
        B1 --> B2[AUTHENTICATE]
        B2 --> B3{ }
        B3 -- CORRECT CREDENTIALS --> B4[CHECK BUS AVAILABILITY]
        B3 --> B1
        B4 --> B5{ }
        B5 -- ROUTES AVAILABLE --> B6[CALCULATE ETA]
        B5 --> B1
        B6 --> B7[SHOW FINAL BUS STATUS]
        B7 --> B8((FINAL PAGE))
    end

    subgraph Computer
        C1[LOGIN] --> C2[SEND CREDENTIALS]
        C2 --> C3[SEARCH ROUTES OR STOPS]
        C3 --> C4[REQUEST BUS ETA]
        C4 --> C5[VIEW ETA & TRIP PLAN]
        C5 --> C6[BOARD BUS]
        C5 --> C7{ }
        C7 -- ERROR CHECK --> C8(( ))
        C7 -- SUCCESS --> C9(( ))
    end

    A3 --> B1
    B1 --> C1
    C4 --> B4
    C6 --> B8
    C9 --> B8
  
```

## Swimlane Segmentation

- **Authority:** Responsible for maintaining route and schedule data.
- **App Backend:** Core system responsible for authentication, data validation, and ETA computation.

- **Commuter:** End user requesting and consuming real-time transport information.
- This structure clearly delineates responsibility boundaries and operational roles within the system.

### Authority Workflow

- **Update Schedules/Routes**
  - The Authority initiates the process by updating the transport schedule, the bus routes, or related operational configurations.
- **Update Bus GPS**
  - Post updating the schedule, the Authority triggers a configuration push to bus GPS units, ensuring that they operate with the most current routing and timing information.
- **Push to Backend**
  - Once updated, the revised configurations are sent to the App Backend. This ensures that ETA computation and route availability validation rely on correct operational data.

### Commuter Workflow

- **Login**
  - The Commuter begins by authenticating within the system.
  - If login fails, the activity flow terminates in the *Fail* state.
  - If successful, the Commuter proceeds to the search activity.
- **Search for Bus/Route**
  - The commuter requests information about a specific bus or route after successful login. This request transitions control to the App Backend for processing.

## **5. App Backend Workflow**

- **Authenticate**
  - The Backend validates commuter credentials.

- Upon failure, the flow is redirected to the commuter's *Fail* activity.
- Upon success, the flow proceeds to route availability verification.
- **Check Bus/Route Availability**
  - The Backend checks whether the requested bus or route exists and is operational.
  - If unavailable, the commuter is returned an *Error* state.
  - If available, the backend attempts to retrieve real-time GPS data.
- **Fetch Real-Time GPS**
  - The Backend requests location telemetry from the bus's GPS unit.
- **Error Handling**
  - At any backend decision node:
  - If processing fails,
  - or validation conditions are unmet,
  - the system displays an *Error* message to the commuter.

## 6. Commuter ETA Interaction and Trip Planning

- **View ETA & Plan Trip**
  - If valid ETA information is successfully computed or fallback information is available, the commuter views the results and plans their commute.
- **Error/No Info Condition**
  - If an error or missing data is encountered:
  - The commuter receives an *Error* notification.
  - The commuter may retry or terminate the session.
- **Success and Boarding**
  - If the commuter is satisfied with the information:
  - They proceed to *Board Bus*, marking successful completion of the flow.

## Chapter 6: UI Design with Screenshots

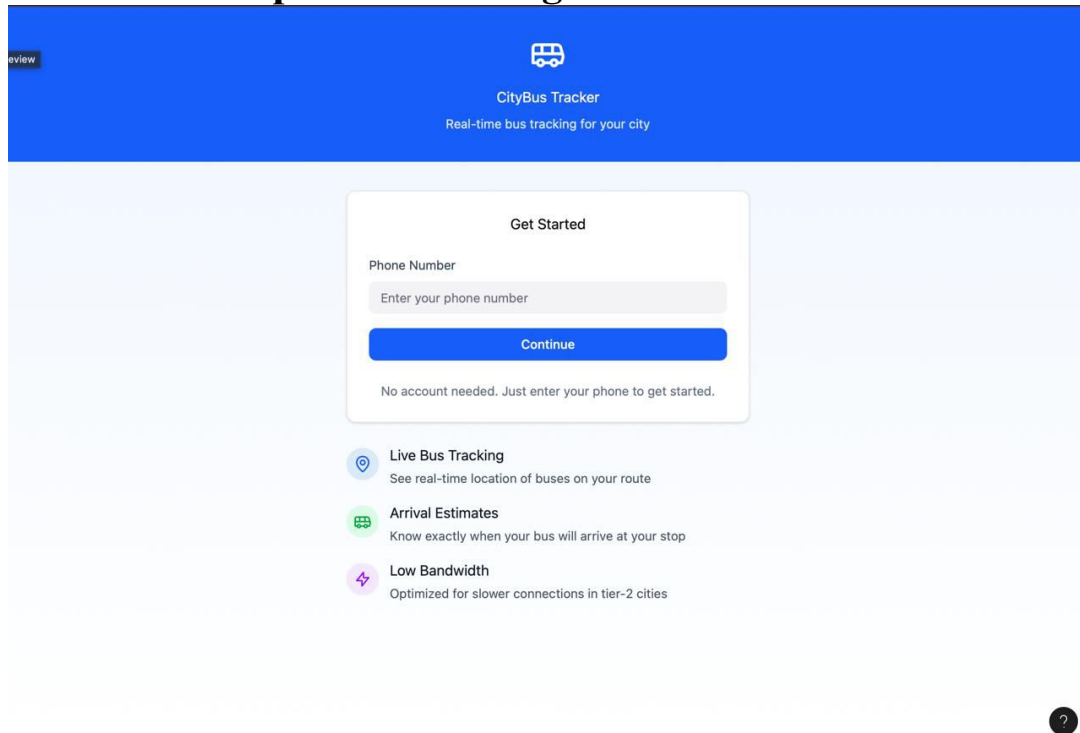


Fig: Login Page

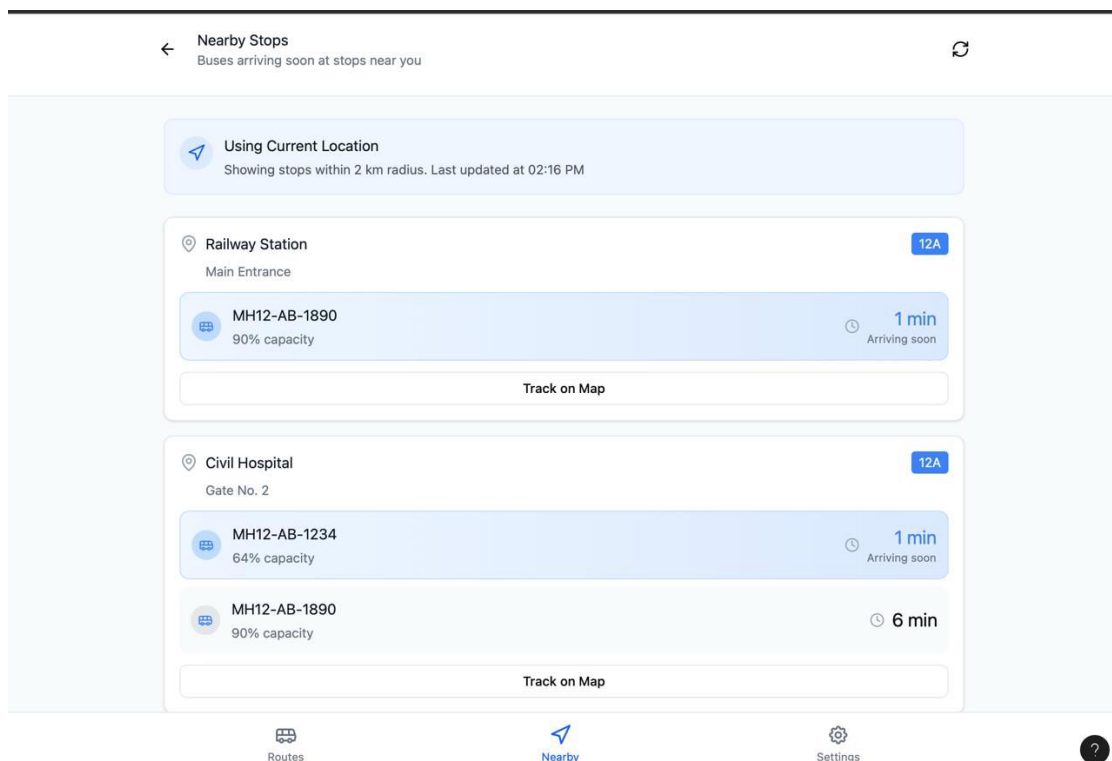


Fig:Bus Tracker

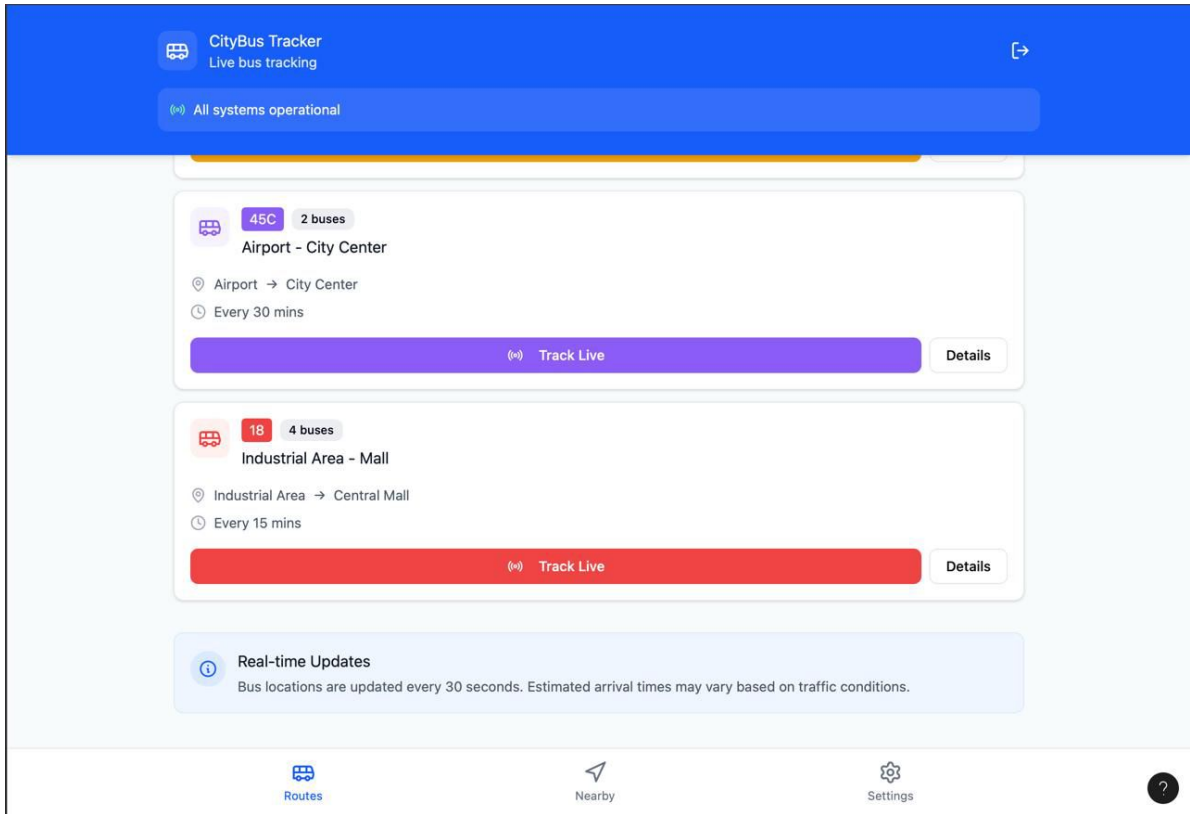


Fig: Live Tracker

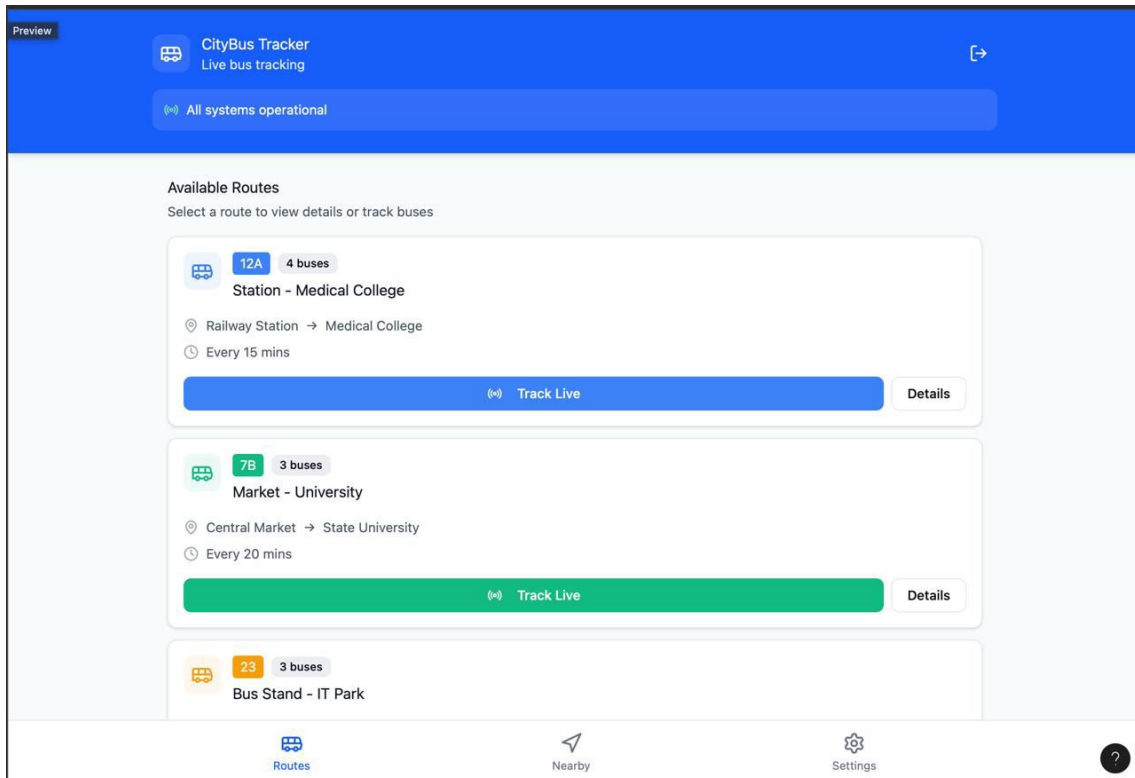


Fig: List Of Buses

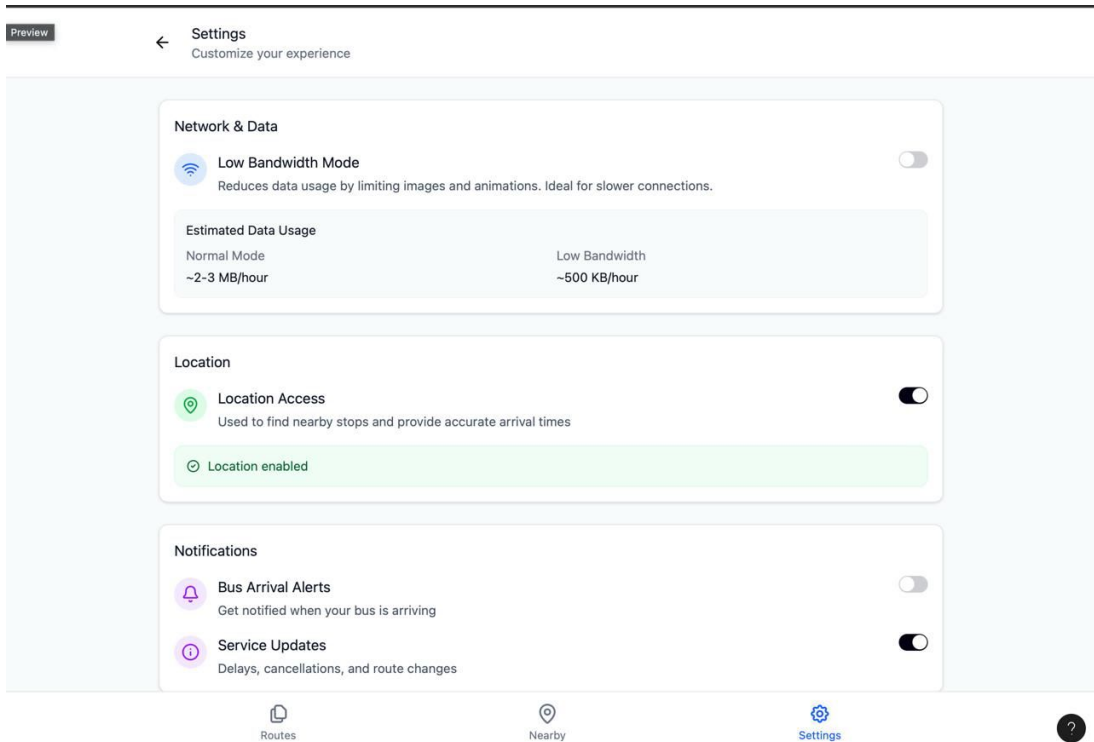


Fig: Settings Page

## **Conclusion**

The Public Transportation System (PTS) presents a comprehensive and technologically robust solution to the longstanding challenges associated with public transportation in growing urban environments. Traditional commuting patterns are often hindered by uncertainty, lack of visibility, inaccurate schedules, and inefficiencies in communication between commuters, transport authorities, and bus operators. Through the integration of GPS-based telemetry, real-time data processing, automated ETA computation, and a user-friendly mobile interface, this project successfully bridges the information gap and significantly enhances the public transport experience.

The system's design demonstrates a strong alignment with modern software engineering principles. Core functionalities—such as commuter authentication, route search, live bus tracking, and notification delivery—are supported by a well-structured architecture comprising entity models, repositories, services, and communication modules. The implementation of state models, use case diagrams, sequence diagrams, and activity diagrams further validates the logical flow and operational integrity of the system. These diagrams collectively illustrate how real-time data is acquired, processed, and distributed to deliver actionable insights to end users. Additionally, the grievance and issue-resolution workflows ensure that operational anomalies are managed effectively, promoting transparency and accountability within transport administration.

In conclusion, the Public Transportation System not only addresses the inefficiencies prevalent in conventional public transportation but also sets the foundation for smarter urban mobility. By enhancing commuter satisfaction, enabling data-driven fleet management, and improving overall public transport reliability, PTS brings measurable value to both users and authorities. As cities continue to expand, such systems will play an essential role in shaping sustainable, connected, and intelligent transportation ecosystems. Future enhancements may include predictive analytics, integration with multimodal transport networks, and AI-driven demand forecasting, further extending the capabilities and societal impact of the system.